The project requires a production ready NodeJS application which has :

1. Infrastructure as Code (IaC)
2. CI/CD Pipelines
3. Containerization
4. AWS Deployment
5. Automation Scripts
6. Security Best Practices

Infrastructure is provisioned using Terraform :

VPC

Subnets

Internet Gateway

Security Groups

ECS Cluster

ECR Repository

IAM Roles

Secrets Manager

CloudWatch Logs

S3

DynamoDB

IAM Setup

Issue Encountered : Terraform failed due to insufficient permissions .

ecr:CreateRepository

iam:CreateRole

secretsmanager:CreateSecret

Since required managed policies were not available, a custom inline policy was created.

Also I used additions policies :

AmazonEC2FullAccess
AmazonECS_FullAccess
CloudWatchLogsFullAccess
AmazonS3FullAccess
AmazonDynamoDBFullAccess

Terraform Setup

Initialize

For initialization I made use of : cd infra and terraform init .

Validate : For validating I made use of terraform validate

Plan : For planning I made use of terraform plan -var-file=envs/dev.tfvars

Docker Setup

I then setup docker for which I built my image using

cd app

docker build -t taski-nodejs-app .

And then for running the image I used : docker run -p 3000:3000 taski-nodejs-app

CI/CD Pipeline

The pipeline includes :

Source trigger (push/PR)

Install dependencies

Lint

Unit tests

Security scan (npm audit + Trivy)

Docker build

Push to ECR

Deploy to ECS

Wait for stable deployment

To run the tests : Use npm test

The automation scripts include :

deploy.sh : deploy ECS service + health check

rollback.sh : revert to previous version

health-check.sh : to validate app health

The Security Best Practices that I Implemented were that

No secrets were stored inside the code , AWS Secrets Manager was used , IAM least privilege was implemented .

Along with that Docker image scanning was enabled and Terraform remote state secured .